

---

---

# COMP2511

Tutorial 1

---

---

# Introduction

- Hello! I'm James (he/him).
- Did a CS degree, am currently doing a secondary teaching degree and want to do medicine in a few years!
- Some of my interests: Video games, tea, pandas.
- This is a classroom! Please ask questions or ask me to re-clarify anything when you need.
- These tutorials are designed to be accessible without watching the lecture content beforehand, but you will get the most out of them if you did watch the lectures!

# Icebreakers

- Please introduce yourself, your year/degree and anything you would like to share!
  - fun facts about yourself
  - anything you did during the holidays (doing nothing counts!)
  - any recent obsessions (music, games, hobbies)
  - your go-to food/drink spot on campus
  - how many hours of sleep you got last night

# Welcome to COMP2511!

- In previous courses, you became more proficient and confident in your abilities as **programmers**.
  - COMP1531: Introduction to working on a larger-scale project as a team
  - COMP2521: Introduction to algorithmic problems and taking efficiency into account
- In this course, the emphasis is on writing code that not only works but is also easy to work with.

# Assessments

- **Labs (15%)** - 8 labs, each *manually marked* out of 10. Your overall lab mark is out of 70 (take sum of all marks), leaving a buffer for 10 marks.
- **Assignment 1 (15%)** - **individual** assignment, where you will build a system from scratch using key concepts from Weeks 1-4.
- **Assignment 2 (20%)** - **individual** assignment split into two parts. Part 1 focuses on **refactoring** a large existing codebase. Part 2 focuses on analysing the requirements of a large-scale software system.
- **Final Exam (50%)** - **40% hurdle**. Approximately 50% of the exam will be very similar in style to lab exercises and tutorial examples.

# What is 'good' design?

- What are some things that we can all agree are desirable or NOT desirable in code?
  - We should write code that adheres to widely used **conventions** for readability (style).
  - We should try to write code that is as simple and easy to understand as possible.
  - It is convenient to use high-level constructs, rather than implementing low-level details or worrying about concrete implementation details (**abstraction**).
  - Different functionalities should be separated into individual modules that work together (**modularisation**).
  - If we need to add new things or make modifications, we should be able to do so without risking breaking the existing code (**extensibility**).
- The last three will be particularly important in this course!

# Object-Oriented Programming

- During the first half of the term, we will be focusing on **object-oriented programming** (OOP).
- A **class** is a user-defined type that owns data **and** methods (functions) that manipulate that stored data.
- An **instance** of a class (i.e. a variable of that type) is referred to as an **object**.
- In OOP, logic is organised around defining different classes and having them interact with each other.

# Blueprint vs. Product

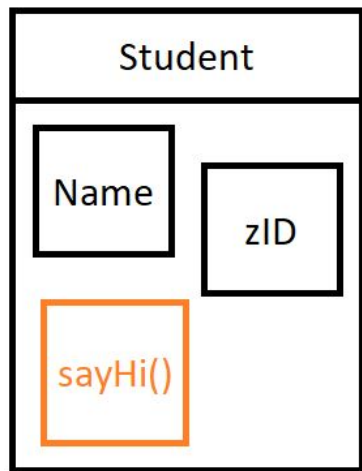


A blueprint of a house; the *idea* that informs what components the house will have and how it will be structured.

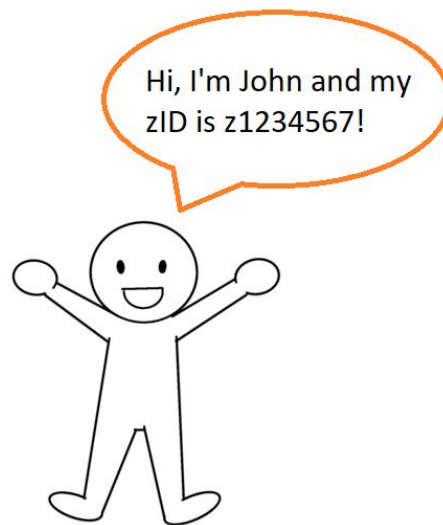


The actual house; the physical *realisation* of the blueprint.

# Class vs. Object



A class; the template/blueprint to create an object. Name/zID are **fields**, sayHi() is a **method**.



An object; an **instantiation** of the Student class. This object has its own name, zID and method to introduce itself.

# Key Terms in Object-Oriented Programming

- We will be exploring how we can apply OOP to design larger-scale **extensible**, **maintainable** and **reusable** systems.
- Key Terms
  - **encapsulation** - grouping data and the mechanisms that act on that data together; this is facilitated through the use of classes. The fields and methods of a class should be very closely tied together.
  - **abstraction** - hiding away unnecessary details of how things are implemented and only exposing the essential features or functionalities to users, i.e. you just need to know the *what*, and not the *how*.
  - We will look at **inheritance** and **polymorphism** next week, when we look at *subclasses*.

# Tech Stack

- We will use **Java** for all code in this course (specifically, Java 21). We do not assume any prior experience with Java!
- Like C, you need to declare static types when you make a variable.
- Unlike C, Java cleans up memory automatically!
  - In *almost* all cases, you will not have to handle any memory leaks.
- All code in Java has to be written inside of a class.
- **Gradle** is a tool used for dependency and build management.
  - You do not have to interact with Gradle outside of using commands listed in the specs. Treat it as a black-box to compile your projects!

# Important Terminology

- **Access modifiers**

- Applied to class fields and methods, and dictates which other classes can access/use it.
- **public** - *all* other classes can access this field/method
- **private** - *no* other classes can access this field/method
- e.g. **public int x**; declares an int stored inside of a class that all other classes can access.

- **Constructors**

- Methods that create an instance of a class, given a list of parameters.
- They are declared like typical methods, with the method name being the name of the class itself.
- e.g. a constructor for a class named Student with fields name and zID can be declared as **public Student(String name, String zID) { ... }**

- **new**

- The Java equivalent of **malloc**. This keyword should be followed by a class constructor and allocates heap memory to create an instance of that class.
- e.g. **Student s = new Student("John", z1234567);**

# Important Keywords

- **Instance fields and methods**

- **Instance fields** belong to particular objects and differ across objects. For example, a student's name and zID are instance fields, since different students may have a different name and zID.
- **Instance methods** are invoked (called) from concrete objects. For example, if **st** is of type Student and **sayHi()** is a method in the Student class, it can be invoked by doing **st.sayHi()**.

- **static**

- A static field or method *belongs to a class itself*, rather than being tied to concrete instances of the class (opposite of instance fields/methods).
- They are basically “global” across all instances of a class.
- **Example:** Constants such as  $\pi$  should be static, since its value is the same across all instances of a class (different objects don't have a different value of  $\pi$ ).

- **this**

- Refers to the actual object the method was invoked from (can only be used in instance methods).
- Useful for when you need to make reference to a class field that **shares the same name as a local variable**. Without **this**, Java will prioritise the variable that is **local** in scope.
- `public Circle(String name, String zID) { this.name = name; this.zID = zID; }`

# Live Coding

- HelloWorld.java
  - Write a program with a `main` method that prints out "Hello World!" to the terminal and run it, then push the code onto git.
- Sum.java
  - Write a program that uses the `Scanner` class to read in a line of numbers separated by spaces, and sums them.
- Shouter.java
  - Write a class that stores a message and has methods for getting the message, updating the message and printing it out in all caps. Write a `main` method for testing this class.

# Quiz Questions

- What is the difference between a class and an object?
- How do you declare the main method in Java?
- How do you print output in Java?
- How do you receive input in Java?
- What is the difference between instance and static fields/methods?